

Reviewer Pack — Covering Radius Bound on Set System Discrepancy

Entry 4ee94497fe16 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 17:02:37 UTC

Covering Radius Bound on Set System Discrepancy

Entry ID: 4ee94497fe16 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 17:02:37 UTC

1. Conjecture

Field A (mathematical branch): Geometry of Numbers **Field B** (complexity object): Communication Complexity

Statement:

For any set system S on n elements, the discrepancy $\text{disc}(S)$ is bounded by the covering radius of the integer lattice $\mathbb{Z}^n$ scaled by the volume of the convex hull of the characteristic vectors of S .

Rationale (proposer's reasoning):

Geometry of numbers provides tools to analyze lattice coverings, which can quantify how well points in space are approximated by lattice points. This connects to communication complexity via discrepancy, where the covering radius reflects the 'spread' of set system representations, influencing lower bounds on communication protocols.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2d40442903abdb28

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def dot_product(v1, v2):
    return sum(x * y for x, y in zip(v1, v2))

def norm(v):
    return math.sqrt(sum(x**2 for x in v))

def matrix_multiplication(A, B):
    return [[dot_product(row, col) for col in zip(*B)] for row in A]

def gaussian_elimination(A, b):
    n = len(b)
    M = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = max(range(i, n), key=lambda x: abs(M[x][i]))
        M[i], M[max_row] = M[max_row], M[i]
        factor = M[i][i]
        for j in range(i, n + 1):
            M[i][j] /= factor
        for k in range(n):
            if k != i:
                factor = M[k][i]
                for j in range(i, n + 1):
                    M[k][j] -= factor * M[i][j]
    return [row[-1] for row in M]

def linear_independence(vectors):
    A = vectors
    b = [0] * len(vectors)
    try:
        gaussian_elimination(A, b)
        return True
    except ZeroDivisionError:
        return False

def convex_hull(points):
    def orientation(p, q, r):
        val = (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])
        if val == 0:
```

```

        return 0
    elif val > 0:
        return 1
    else:
        return 2

def distance(p, q):
    return math.sqrt((p[0] - q[0])**2 + (p[1] - q[1])**2)

if len(points) < 3:
    return points

lower = []
for p in points:
    while len(lower) >= 2 and orientation(lower[-2], lower[-1], p) != 2:
        lower.pop()
    lower.append(p)

upper = []
for p in reversed(points):
    while len(upper) >= 2 and orientation(upper[-2], upper[-1], p) != 2:
        upper.pop()
    upper.append(p)

return lower[:-1] + upper[::-1]

def covering_radius(n):
    # Approximate the covering radius using a Monte Carlo method
    points = [[random.gauss(0, 1) for _ in range(n)] for _ in range(1000)]
    hull = convex_hull(points)
    max_distance = 0
    for point in hull:
        distances = [distance(point, p) for p in points]
        max_distance = max(max_distance, min(distances))
    return max_distance

def discrepancy(S):
    n = len(S)
    characteristic_vectors = [[1 if i in subset else -1 for i in range(n)] for subset in S]
    A = matrix_multiplication(characteristic_vectors, characteristic_vectors)
    b = [sum(1 for subset in S if i in subset) for i in range(n)]
    try:
        solution = gaussian_elimination(A, b)
        return max(abs(x) for x in solution)
    except ZeroDivisionError:
        return float('inf')

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    S = [set(random.sample(range(n), k=random.randint(1, n))) for _ in range(100)]
    disc_S = discrepancy(S)
    Z_n_covering_radius = covering_radius(n)
    scaled_radius = Z_n_covering_radius * (2 ** (n / 2)) / math.sqrt(math.pi * n)

    return {

```

```

        "metric_name": "discrepancy_bound",
        "metric_value": disc_S,
        "instances_tested": len(S),
        "conjecture_holds": disc_S <= scaled_radius,
        "counterexample": "" if disc_S <= scaled_radius else f"Discrepancy {disc_S} exceeds bound"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8c996cbf.py", line 124, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8c996cbf.py", line 108, in run_trial
    Z_n_covering_radius = covering_radius(n)
                          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8c996cbf.py", line 88, in covering_radius
    distances = [distance(point, p) for p in points]
                  ~~~~~^
NameError: name 'distance' is not defined. Did you mean: 'distances'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined 'distance' function, preventing data collection. | next: Implement missing distance metric and revalidate conjecture with corrected code

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	114093
2	propose	ollama_remote	qwen3:8b	0	0	147627
3	preregistration	ollama_remote	qwen3:8b	0	0	24876
4	novelty	ollama_remote	qwen3:8b	0	0	20922
5	novelty	ollama_remote	qwen3:8b	0	0	15204
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19768
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13770
8	judge	ollama_remote	qwen3:8b	0	0	13782

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 370042 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/4ee94497fe16.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4ee94497fe16.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4ee94497fe16.tar.gz (if generated)